

STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems in Production

Anonymous ACL submission

Abstract

Production AI agent systems increasingly suffer from *architectural lock-in*: early commitment to a single interaction protocol, a fixed tool-integration path, and a static user model makes deployed agents hard to retarget when requirements, client surfaces, or emerging standards shift. We describe **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent), an architecture designed for an enterprise deployment that must simultaneously serve human end-users, other agents, web UIs, and commerce backends. STEM Agent unifies five interoperability protocols—A2A, AG-UI, A2UI, UCP, and AP2—behind a single gateway, isolates domain knowledge behind the Model Context Protocol (MCP), and continuously adapts per-user behavior through a Caller Profiler that learns preferences across 21 behavioral dimensions in four categories. Recurring interaction patterns *crystallize* into reusable skills through a maturation lifecycle, and a development-time differentiation layer turns the generic agent into domain variants via a persona schema, scoped rules, and an MCP server wrapper. We report lessons from integrating five interoperability standards in one gateway, discuss compromises, and outline open problems. The implementation is validated by a 422-test integration suite that runs in under 3 s.

1 Introduction

Teams deploying LLM-based agents inside a large enterprise quickly discover that the hard problems are not which reasoning prompt to use, but how the agent connects to the rest of the business: which protocol it speaks to other agents, which payment or commerce standard it must satisfy, how it streams partial progress to a web UI, and how it remembers what it learned from each caller. Most popular agent frameworks (Tran et al., 2025; Ferrag et al., 2025) commit early to one answer for each

of these, producing agents that are hard to retarget when an additional consumer or protocol appears. Orogat et al. (2026) confirm that framework choice alone can drive $100\times$ latency differences and large accuracy gaps. At the same time, the interoperability landscape is fragmenting (Ehtesham et al., 2025; Li and Xie, 2025): MCP has emerged as a *de facto* tool protocol, A2A is gaining traction for agent-to-agent delegation, and additional standards for UI streaming, commerce, and agent payments are entering production.

This paper reports on the architecture and deployment experience of STEM Agent, a system built for the authors’ organization to serve these needs without repeatedly rewriting core logic. The design principle is biological: an undifferentiated agent core *differentiates* into specialized protocol handlers, tool bindings, and memory subsystems, which *compose* into a functioning system. Practically this means every external boundary—protocols, tools, user models, and domains—is a pluggable differentiation point rather than a hard-wired choice.

Contributions. We describe (i) a single gateway exposing five interaction protocols (A2A, AG-UI, A2UI, UCP, and AP2), of which A2A and AG-UI follow published specifications and UCP and AP2 are proposed herein to fill commerce gaps we hit in deployment; (ii) a Caller Profiler that learns preferences across four categories and 21 dimensions using bounded exponential-moving-average updates, coupled to ten self-tunable behavior parameters; (iii) an MCP-native tool layer that separates domain knowledge from meta-reasoning; (iv) a skills subsystem in which recurring interaction patterns crystallize into reusable skills that mature or undergo apoptosis based on outcomes; (v) a development-time differentiation mechanism that produces domain-specific agent variants from a persona schema; and (vi) a 422-test integration

suite and a report on what this design cost and bought us.

This is an industry-track paper: novelty is in integration and deployment discipline, not in a new model or algorithm. We flag claims that rest on internal measurements and explicitly separate them from behaviors validated by automated tests.

2 Related Work

Multi-agent frameworks. Surveys (Tran et al., 2025; Ferrag et al., 2025) document a large ecosystem of LLM-based agent frameworks—AutoGen (Wu et al., 2023), MetaGPT (Hong et al., 2023), CAMEL (Li et al., 2023), CrewAI (Moura, 2023), LangChain (Chase, 2022), and LangGraph (LangChain AI, 2024)—each optimizing for one coordination pattern and typically one protocol. Orogat et al. (2026) show that this architectural commitment dominates observable performance. Dang et al. (2025) propose evolving orchestration; Drammeh (2025) demonstrate deterministic multi-agent orchestration for incident response. Unlike frameworks that bind each agent to a single protocol endpoint, every STEM gateway mounts five heterogeneous handlers behind one shared middleware stack, and treats per-caller adaptation as a first-class architectural surface.

Agent communication protocols. Ehtesham et al. (2025) compare MCP, ACP, A2A, and ANP and argue MCP and A2A are complementary (vertical tool access vs. horizontal messaging). Li and Xie (2025) identify schema translation and lifecycle management as the hard parts of combining A2A and MCP, which matches our experience. Jeong (2025) further study MCP×A2A. Security has received active attention (Habler et al., 2025; Anbiaee et al., 2026; Sarkar and Sarkar, 2025). Chang et al. (2025) address decentralized agent discovery. We are not aware of a deployed system that unifies these five protocols behind a single gateway.

MCP tools and adaptation. Benchmarks for the Model Context Protocol (Anthropic, 2024) (Luo et al., 2025; Fan et al., 2025) highlight that even strong LLMs struggle on real MCP servers, and Hasan et al. (2026) find that 97% of MCP tool descriptions have quality issues. Lumer et al. (2025) propose dynamic synchronization; Jayanti and Han (2026) and Schlapbach (2026) extend MCP with shared context and schema-guided convergence.

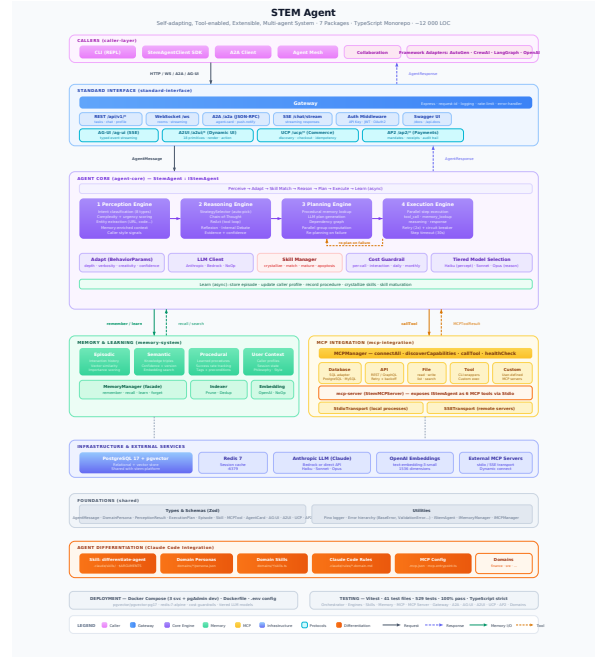


Figure 1: STEM Agent architecture. Callers reach the gateway, which mounts five protocol handlers (A2A, AG-UI, A2UI, UCP, AP2) sharing a common middleware stack. The Agent Core’s cognitive pipeline consumes MCP tools and a four-type memory system; the Differentiation Layer produces domain-specific variants.

Agentic reasoning surveys (Wei et al., 2026; Alomrani et al., 2025; Li et al., 2025) catalog ReAct, Reflexion, tree-of-thought, and debate; STEM Agent uses these as interchangeable strategies selected per task. On memory and self-adaptation, Ai et al. (2025) benchmark continual learning, Gallego (2026) treat memory as a learnable capability, and Yadav et al. (2026) present hierarchical memory for delivery agents—parallels we build on for our four-type memory system.

3 Architecture Overview

STEM Agent is organized into five layers plus a cross-cutting differentiation surface (Table 1, Figure 1). Layer 3 (Agent Core) is the undifferentiated core; Layer 2 (protocol handlers) and Layer 5 (MCP tools) provide the external surfaces through which it differentiates; Layer 4 (Memory) supplies the persistent state that drives adaptation. The Differentiation Layer bundles the artifacts—domain personas, skills, rules, and MCP configuration—needed to transform the generic agent into a domain-specific variant.

Layer	Component	Key Count
1	Caller / User Layer	4 adapters
2	Standard Interface (Gateway)	5 protocols
3	Agent Core	5 engines
3.5	Security Layer	4 IAM plugins
4	Memory System	4 memory types
5	MCP Integration + Server	2 transports
⊥	Differentiation Layer	personas/skills/rules

Table 1: Five-layer architecture with the cross-cutting Differentiation Layer.

Condition	Strategy
Tool-requiring request	ReAct
Complexity > 0.8	Reflexion
Analysis or creative request	Internal Debate
Otherwise	Chain-of-Thought

Table 2: Deterministic strategy selector used by SELECTSTRATEGY.

Cognitive pipeline. Each request flows through eight phases: PERCEIVE (intent into 10 categories, complexity into three levels, entities, sentiment, urgency); ADAPT (load caller profile, adjust behavior parameters); MATCHSKILLS (short-circuit if a committed or mature skill fires); SELECTSTRATEGY and REASON (selector in Table 2, with ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), Internal Debate (Du et al., 2023), and Chain-of-Thought (Wei et al., 2022)); PLAN (MCP tool selection with parallel steps); EXECUTE (retries with default 2, circuit breaker at 3 consecutive failures); FORMAT; and an asynchronous LEARN phase that updates the profile, records skill outcomes, and attempts crystallization.

Implementation. The system is a TypeScript monorepo with six workspace packages covering shared schemas (154+ Zod types), agent core, the gateway and handlers, MCP client and server layers, memory, and caller utilities. The gateway runs on Express.js 5 and mounts each protocol handler via a uniform createRouter() method, so that adding a new protocol affects only the handler and one line in the gateway. Authentication, rate limiting, request correlation, and structured logging are shared middleware.

4 Protocol Gateway

The single most load-bearing design decision in STEM Agent is that all five protocols share one process and one cognitive pipeline. Table 3 summarizes them.

External protocols. A2A implements the Linux Foundation v0.3.0 specification over JSON-RPC 2.0 with tasks/send, tasks/sendSubscribe (SSE), tasks/get, and tasks/cancel; agent cards are served at /.well-known/agent.json. AG-UI streams cognitive-pipeline events (TEXT_MESSAGE_START, REASONING_MESSAGE, TOOL_CALL_START, STATE_SNAPSHOT, RUN_FINISHED) to frontends via SSE, giving web UIs fine-grained progress without a custom streaming format. A2UI exposes a flat adjacency-list UI model with 16 primitives (text, button, card, list, input, toggle, ...) so that the agent can assemble layouts without a fixed widget hierarchy.

UCP (Universal Commerce Protocol). UCP formalizes a checkout-session lifecycle—create, retrieve, complete—under mandatory idempotency headers (Idempotency-Key, Request-Id, UCP-Agent). An idempotency cache prevents duplicate session creation when a caller retries. UCP is proposed here because we could not find an open commerce-checkout protocol that treated the agent, rather than a browser, as the primary client.

AP2 (Agent Payments Protocol). AP2 implements a three-phase payment lifecycle: Intent Mandate → Payment Mandate → Payment Receipt. Payments below a configurable threshold can auto-approve; all transitions are appended to a per-intent audit trail. AP2 is the counterpart to UCP for authorization and is explicitly designed around mandate semantics appropriate to autonomous agents.

Integration lessons. Two lessons stand out. First, the createRouter() uniformity is worth enforcing even when a protocol seems to “want” global state: making each handler self-contained reduced cross-protocol bugs to near zero during the addition of AP2 after the other four. Second, idempotency is not a nice-to-have but the actual hard part—UCP’s header contract and AP2’s mandate IDs each took more review cycles than the happy-path code.

5 Caller Profiler and Behavior Parameters

Static user models produce either bland responses or hand-tuned personas that drift. STEM Agent continuously learns a Caller Profile across four categories, building on a long line of user-modeling and generative-agent work (Park et al., 2023): *philosophy* (8 dimensions), *principles* (4), *style* (5),

Protocol	Endpoint	Transport	Role	Use case
A2A	POST /a2a	JSON-RPC 2.0	External (Linux Foundation A2A v0.3.0)	Agent-to-agent task delegation, messaging, agent-card discovery
AG-UI	POST /ag-ui	SSE	External	UI streaming; maps pipeline phases to typed events
A2UI	POST /a2ui/*	SSE + REST	External	Dynamic UI composition with 16 component primitives
UCP	POST /ucp/*	REST	Proposed	Commerce checkout sessions with idempotency
AP2	POST /ap2/*	REST	Proposed	Mandate-based payments with audit trail

Table 3: Protocols behind the unified gateway. A2A, AG-UI, and A2UI follow external specifications; UCP and AP2 are proposed in this work.

and *habits* (4), for a total of 21.¹

EMA update. Each dimension is updated with an exponential moving average,

$$v_{t+1} = (1 - \alpha) v_t + \alpha s_t, \quad (1)$$

with learning rate $\alpha = 0.1$. Smaller α produced sluggish adaptation in preliminary testing; larger α oscillated. Per-caller locks prevent concurrent update conflicts.

Confidence gating. Profile confidence ramps with interaction count as $\text{conf}(n) = 1 - \exp(-0.1 n)$, reaching approximately 0.9 at $n=20$. Below $n=5$ the system prefers signals extracted from the current message; beyond that it blends profile and signal by confidence weight.

Self-tunable behavior. Ten behavior parameters are adjusted during ADAPT: reasoning depth (default 3), exploration vs. exploitation (0.3), verbosity (0.5), confidence threshold (0.7), tool-use preference (0.5), creativity (0.5), proactive suggestion (on), self-reflection frequency (every 5 steps), max plan steps (10), and memory retrieval breadth (10). The parameters feed the reasoning strategy selector and the planning budget, so “how hard to think” is a function of both the task and the caller.

6 Differentiation: Skills and MCP-Native Tools

Agents in production accumulate patterns. Rather than letting that accumulation sit in logs, STEM Agent crystallizes it.

¹The 21 dimensions are: *philosophy*—pragmatism vs. idealism, simplicity vs. completeness, risk tolerance, innovation orientation, depth vs. breadth, theory vs. practice, autonomy preference, stated values; *principles*—correctness vs. speed, documentation importance, testing emphasis, security-mindedness; *style*—formality, verbosity, technical depth, examples preference, structure preference; *habits*—typical session length, iteration tendency, peak hours, common topics.

Skill lifecycle. A skill bundles a trigger (intent patterns, domain tags, entity types), an action sequence (tool chain or procedure steps), and maturity metadata. During LEARN, recent episodes are grouped by action pattern; if ≥ 3 episodes share a common action key and their topic keywords overlap in at least half the group, a *progenitor* skill is created. After $k_c=3$ successful activations with success rate ≥ 0.6 a skill becomes *committed* and may short-circuit REASON→PLAN; after $k_m=10$ it becomes *mature* and receives matching priority. Skills whose success rate falls below 0.3 after ≥ 10 activations undergo *apoptosis* and are removed. These thresholds are our current operating points, tuned against the existing test suite rather than a benchmark distribution. Users can also manually register plugin skills, bypassing progenitor status.

MCP-native tools. All domain capabilities are acquired at runtime through MCP: database access, API integrations, file operations, and vertical tools are discovered and invoked without recompiling the agent. Meta-reasoning rules (strategy selection, parameter tuning, skill lifecycle) deliberately remain in code, not in MCP, because they are properties of the agent, not the domain.

Development-time differentiation. A Domain Persona schema (systemPrompt, preferredStrategy, defaultBehavior overrides, toolAllowlist, requiredMCPServers, forbiddenTopics, domainTags) parameterizes the generic agent. Each domain directory contains the persona, plugin skills registered at committed maturity, and glob-scoped rules consumed by tooling when editing domain files. An mcp-server package wraps the running agent as MCP tools (stem_process, stem_list_skills, stem_recall_memory, stem_agent_card,

stem_health, stem_get_persona) so other MCP clients can invoke a differentiated variant. A single slash-command skill (/differentiate-agent \$DOMAIN) assembles all of the above and verifies the result. In the authors’ experience, creating a new domain variant with /differentiate-agent takes on the order of minutes, compared with the multi-day effort previously required to fork and rewire an agent codebase; we do not have a controlled measurement.

7 Memory System

Four memory types, informed by recent agent-memory work (Ai et al., 2025; Gallego, 2026; Yadav et al., 2026), sit behind a Memory Manager facade:

- **Episodic.** Interaction episodes with vector embeddings and importance scores (PostgreSQL + pgvector).
- **Semantic.** Knowledge triples (subject, predicate, object) in a concept graph, populated by pattern extraction from episodic memory.
- **Procedural.** Successful strategies and tool-usage patterns used to prime planning on recurring tasks.
- **User context.** Per-caller profile and session history, with an explicit forget-me path for regulatory compliance.

Consolidation moves information episodic→semantic and episodic→procedural as history grows, and low-utility entries are pruned under capacity bounds. Together with deduplication in the semantic store, this is designed for sub-linear growth under sustained interaction; exact growth characteristics under real traffic are left to a future measurement study.

8 Deployment Experience

This is a deployment-ready architecture report rather than a deployed-at-scale measurement study: the numbers below come from the repository test suite. Task-completion benchmarks, traffic numbers, and user studies are explicitly not claimed, and we flag this mismatch with the Industry Track’s usual expectations in the Limitations.

Test suite. The suite comprises **422 tests across 40 files** (Vitest), passing at 100% with total runtime of approximately **2.86 s** on a single laptop. Every architectural seam is covered by at least one test: per-engine unit tests (perception, reasoning, planning, execution, skills), protocol handlers (A2A JSON-RPC compliance, AG-UI event sequences, A2UI component validation, UCP idempotency, AP2 audit trails), memory stores (episodic, semantic, procedural, user-context), MCP client and server integration tests (via InMemoryTransport), domain persona and rule schema validation, security middleware (JWT, OAuth2, API-Key), framework adapters, and gateway end-to-end tests. A test count is a weak correctness signal on its own; what the suite actually validates is that each seam in Figure 1 can be exercised in isolation and that adding a new protocol or persona does not silently break another.

Architectural overhead. Each protocol handler adds a thin routing and serialization layer on top of the shared cognitive pipeline: A2A contributes JSON-RPC envelope parsing and task state; AG-UI contributes SSE channel setup; A2UI adds component-tree construction and layout validation; UCP adds idempotency lookup and checkout state; AP2 adds mandate validation and audit writes. Under the shared LLM-mocked test harness, handler-specific overhead is dominated by serialization and middleware traversal rather than cognition. Formal latency benchmarks under controlled conditions are future work.

Integration patterns. Four framework adapters (AutoGen, CrewAI, LangGraph, OpenAI Agents SDK) translate external conventions to the internal message format, so a team already invested in one of those frameworks can call STEM Agent as an A2A peer without rewriting callsites. The IAM layer is plugin-based (JWT, OAuth2, SAML, API-Key) with a TTL policy cache; this has been essential for meeting enterprise auth requirements without touching agent logic.

Feature comparison. Table 4 contrasts the STEM surface with prior systems at the level of whether each framework ships a named subsystem for the capability; this is a capability surface, not a performance benchmark. Counts of standardized protocols reflect those with published, externally-maintained specifications.

Framework	Proto.	Adapt	Mem.
AutoGen (Wu et al., 2023)	1	×	✓
MetaGPT (Hong et al., 2023)	1	×	✓
CrewAI (Moura, 2023)	1	×	✓
LangChain (Chase, 2022)	1	×	✓
STEM	5 (3+2 [†])	✓	4 types

Table 4: Capability-surface comparison. Proto. = interoperability protocols; Adapt = per-caller adaptation as a first-class subsystem; Mem. = named memory subsystem (✓) or explicit type count; Skills = emergent skill acquisition; Com. = commerce protocols. [†]UCP and AP2 proposed herein.

What required compromise. Three points are worth naming honestly. (i) Idempotency and audit trails for UCP/AP2 are in-memory by default; a distributed deployment needs an external store and a lock service. (ii) The EMA-based profiler cannot model multi-modal or rapidly non-stationary preferences; we chose simplicity over a mixture model because the deployed signal-to-noise made it unnecessary. (iii) Reasoning-strategy selection is deterministic rather than learned—easier to reason about for reviewers and auditors, but likely suboptimal on task distributions we have not seen.

9 Conclusion

STEM Agent shows that protocol plurality, behavioral self-adaptation, and emergent skills can co-exist in a single production system when domain knowledge is kept behind MCP and user modeling is decoupled from domain logic. The biologically-inspired framing is not decorative: it gives every external surface a uniform place—protocol handler, MCP tool, persona, or skill—so differentiation is a composition rather than a rewrite. Ongoing work includes external measurement studies, learned strategy selection (Alomrani et al., 2025), interoperability with additional protocols (Chang et al., 2025; Nie et al., 2026), embedding-based skill matching, and formal threat modeling for UCP/AP2 (Anbiaee et al., 2026).

Limitations

No end-to-end task benchmark. Our evaluation validates protocol compliance and cross-component integration through a fast test suite. We do not report task-completion scores on a public multi-agent benchmark (e.g., MAFBench (Orogat et al., 2026) or MCP-Universe (Luo et al., 2025)), nor do we report a user study. The feature-

comparison table contrasts capability surfaces, not performance.

Internal numbers only. The 422-test count and sub-3-second runtime come from the repository test suite. We do not claim production latency, throughput, user adoption, or accuracy numbers in this paper because we cannot publish the corresponding internal measurement harness anonymously.

Adaptation model simplicity. The EMA-based Caller Profiler cannot represent non-stationary or multi-modal preference distributions, and confidence gating is a scalar rather than per-dimension signal.

Protocol maturity. UCP and AP2 are proposed here and have no external adoption, formal threat model, or conformance suite. The in-memory idempotency cache and audit trail presume a single-process deployment; horizontal scaling requires external state.

Deterministic strategy selection. Reasoning-strategy selection uses hand-written rules rather than a learned policy and has not been tuned against a benchmark distribution.

Ethical Considerations

The Caller Profiler stores per-user behavioral signals. To mitigate privacy and consent risks we (i) expose a GDPR-style forget-me operation that removes all per-caller profile and session state; (ii) keep profile updates reversible (EMA with bounded α so a single misclassified signal cannot dominate); (iii) scope audit logs at the request level so misuse can be traced without exposing content. The commerce protocols (UCP, AP2) handle payment intents; in deployment we require human approval above a configurable threshold and emit a complete per-intent audit trail, but we stress that UCP/AP2 have not undergone formal security review. Any deployer should treat these as first-draft protocols and add domain-specific guardrails. More broadly, agent frameworks that integrate authentication, commerce, and persistent user modeling into one process enlarge the blast radius of a compromise; the pluggable IAM layer and layered architecture are partial mitigations, not a substitute for operational review.

References

Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. MemoryBench: A benchmark for memory and con-

478	tinual learning in LLM systems. <i>arXiv preprint</i>	Idan Habler, Ken Huang, Vineeth Sai Narajala, and	530
479	<i>arXiv:2510.17281</i> .	Prashant Kulkarni. 2025. Building a secure agen-	531
480	Mohammad Ali Alomrani, Yingxue Zhang, Derek Li,	tic AI application leveraging A2A protocol. <i>arXiv</i>	532
481	Qianyi Sun, Soumyasundar Pal, Zhanguang Zhang,	<i>preprint arXiv:2504.16902</i> .	533
482	Yaochen Hu, Rohan Deepak Ajwani, Antonios Valka-	Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Ra-	534
483	nas, and 1 others. 2025. Reasoning on a budget: A	jbahadur, Bram Adams, and Ahmed E. Hassan. 2026.	535
484	survey of adaptive and controllable test-time compute	Model context protocol (MCP) tool descriptions are	536
485	in LLMs. <i>arXiv preprint arXiv:2507.02076</i> .	smelly! towards improving AI agent efficiency with	537
486	Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gun-	augmented MCP tool descriptions. <i>arXiv preprint</i>	538
487	jan Piya, Igor Opushnyev, Ali Ghorbani, and Saj-	<i>arXiv:2602.14878</i> .	539
488	jad Dadkhah. 2026. Security threat modeling for	Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu	540
489	emerging AI-agent protocols: A comparative analy-	Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang,	541
490	sis of MCP, A2A, Agora, and ANP. <i>arXiv preprint</i>	Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang	542
491	<i>arXiv:2602.11327</i> .	Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu,	543
492	Anthropic. 2024. Model Context Protocol spec-	and Jürgen Schmidhuber. 2023. MetaGPT: Meta pro-	544
493	ification. https://modelcontextprotocol.io/	gramming for a multi-agent collaborative framework.	545
494	specification .	<i>arXiv preprint arXiv:2308.00352</i> .	546
495	Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao	Meenakshi Amulya Jayanti and X. Y. Han. 2026.	547
496	Cai, Binbin Chen, Xuan Xie, and Yin Zhang. 2025.	Enhancing model context protocol (MCP) with	548
497	Agent network protocol technical white paper. <i>arXiv</i>	context-aware server collaboration. <i>arXiv preprint</i>	549
498	<i>preprint arXiv:2508.00007</i> .	<i>arXiv:2601.11595</i> .	550
499	Harrison Chase. 2022. LangChain. https://github.	Cheonsu Jeong. 2025. A study on the MCP x	551
500	com/langchain-ai/langchain .	A2A framework for enhancing interoperability of	552
501	Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan,	LLM-based autonomous agents. <i>arXiv preprint</i>	553
502	Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang,	<i>arXiv:2506.01804</i> .	554
503	Xiaoyin Che, Ye Tian, and 1 others. 2025. Multi-	LangChain AI. 2024. LangGraph: A library for building	555
504	agent collaboration via evolving orchestration. <i>arXiv</i>	stateful, multi-actor applications with LLMs. https:	556
505	<i>preprint arXiv:2505.19591</i> .	//github.com/langchain-ai/langgraph .	557
506	Philip Drammeh. 2025. Multi-agent LLM orches-	Guohao Li, Hasan Abed Al Kader Hammoud, Hani	558
507	tration achieves deterministic, high-quality deci-	Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023.	559
508	sion support for incident response. <i>arXiv preprint</i>	CAMEL: Communicative agents for “mind” explo-	560
509	<i>arXiv:2511.15755</i> .	ration of large language model society. <i>Advances in</i>	561
510	Yilun Du, Shuang Li, Antonio Torralba, Joshua B.	<i>Neural Information Processing Systems (NeurIPS)</i> .	562
511	Tenenbaum, and Igor Mordatch. 2023. Improving	Junyan Li, Wenshuo Zhao, Yang Zhang, and Chuang	563
512	factuality and reasoning in language models through	Gan. 2025. Steering LLM thinking with budget guid-	564
513	multiagent debate. <i>arXiv preprint arXiv:2305.14325</i> .	ance. <i>arXiv preprint arXiv:2506.13752</i> .	565
514	Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and	Qiaomu Li and Ying Xie. 2025. From glue-code to	566
515	Saket Kumar. 2025. A survey of agent interoperabil-	protocols: A critical analysis of A2A and MCP in-	567
516	ity protocols: Model context protocol (MCP), agent	tegration for scalable agent systems. <i>arXiv preprint</i>	568
517	communication protocol (ACP), agent-to-agent pro-	<i>arXiv:2505.03864</i> .	569
518	protocol (A2A), and agent network protocol (ANP). <i>arXiv</i>	Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah,	570
519	<i>preprint arXiv:2505.02279</i> .	Pradeep Honaganahalli Basavaraju, and James A.	571
520	Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian	Burke. 2025. ScaleMCP: Dynamic and auto-	572
521	Mo. 2025. MCPToolBench++: A large scale AI	synchronizing model context protocol tools for LLM	573
522	agent model context protocol MCP tool use bench-	agents. <i>arXiv preprint arXiv:2505.06416</i> .	574
523	mark. <i>arXiv preprint arXiv:2508.07575</i> .	Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao,	575
524	Mohamed Amine Ferrag, Norbert Tihanyi, and Mer-	Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo,	576
525	ouane Debbah. 2025. From LLM reasoning to au-	Silvio Savarese, Caiming Xiong, and Junnan Li. 2025.	577
526	tonomous AI agents: A comprehensive review. <i>arXiv</i>	MCP-Universe: Benchmarking large language mod-	578
527	<i>preprint arXiv:2504.19678</i> .	els with real-world model context protocol servers.	579
528	Víctor Gallego. 2026. Distilling feedback into memory-	<i>arXiv preprint arXiv:2508.14704</i> .	580
529	as-a-tool. <i>arXiv preprint arXiv:2601.05960</i> .	João Moura. 2023. CrewAI: Framework for orches-	581
		trating role-playing, autonomous AI agents. https:	582
		//github.com/crewAIInc/crewAI .	583

584	Xiaohang Nie, Zihan Guo, Youliang Chen, Yuan-	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak	638
585	jian Zhou, and Weinan Zhang. 2026. AWCP: A	Shafran, Karthik Narasimhan, and Yuan Cao. 2023.	639
586	workspace delegation protocol for deep-engagement	ReAct: Synergizing reasoning and acting in language	640
587	collaboration across remote agents. <i>arXiv preprint</i>	models. In <i>International Conference on Learning</i>	641
588	<i>arXiv:2602.20493</i> .	<i>Representations (ICLR)</i> .	642
589	Abdelghny Orogat, Ana Rostam, and Essam Man-		
590	sour. 2026. Understanding multi-agent LLM frame-		
591	works: A unified benchmark and experimental analy-		
592	sis. <i>arXiv preprint arXiv:2602.03128</i> .		
593	Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai,		
594	Meredith Ringel Morris, Percy Liang, and Michael S.		
595	Bernstein. 2023. Generative agents: Interactive simu-		
596	lacra of human behavior. In <i>Proceedings of the 36th</i>		
597	<i>Annual ACM Symposium on User Interface Software</i>		
598	<i>and Technology (UIST)</i> .		
599	Anjana Sarkar and Soumyendu Sarkar. 2025. Survey		
600	of LLM agent communication with MCP: A soft-		
601	ware design pattern centric review. <i>arXiv preprint</i>		
602	<i>arXiv:2506.05364</i> .		
603	Andreas Schlapbach. 2026. The convergence of schema-		
604	guided dialogue systems and the model context pro-		
605	toocol. <i>arXiv preprint arXiv:2602.18764</i> .		
606	Noah Shinn, Federico Cassano, Ashwin Gopinath,		
607	Karthik Narasimhan, and Shunyu Yao. 2023. Re-		
608	flexion: Language agents with verbal reinforcement		
609	learning. In <i>Advances in Neural Information Pro-</i>		
610	<i>cessing Systems (NeurIPS)</i> .		
611	Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen,		
612	Quoc-Viet Pham, Barry O'Sullivan, and Hoang D		
613	Nguyen. 2025. Multi-agent collaboration mech-		
614	anisms: A survey of LLMs. <i>arXiv preprint</i>		
615	<i>arXiv:2501.06322</i> .		
616	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten		
617	Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and		
618	Denny Zhou. 2022. Chain-of-thought prompting elic-		
619	its reasoning in large language models. In <i>Advances</i>		
620	<i>in Neural Information Processing Systems (NeurIPS)</i> .		
621	Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning,		
622	Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu,		
623	Xiao Lin, Dongqi Fu, and 1 others. 2026. Agentic		
624	reasoning for large language models. <i>arXiv preprint</i>		
625	<i>arXiv:2601.12538</i> .		
626	Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu,		
627	Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang,		
628	Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadal-		
629	lah, Ryen W. White, Doug Burger, and Chi Wang.		
630	2023. AutoGen: Enabling next-gen LLM applica-		
631	tions via multi-agent conversation. <i>arXiv preprint</i>		
632	<i>arXiv:2308.08155</i> .		
633	Arin Gopalan Yadav, Varad Dherange, and Kumar		
634	Shivam. 2026. Project synapse: A hierarchical		
635	multi-agent framework with hybrid memory for au-		
636	tonomous resolution of last-mile delivery disruptions.		
637	<i>arXiv preprint arXiv:2601.08156</i> .		